[3]

Making a Commit

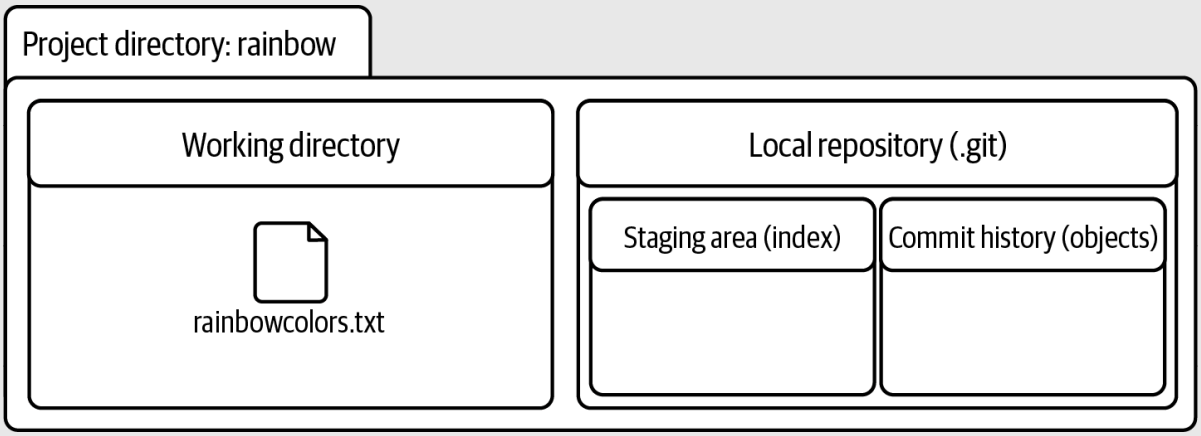
In the last chapter, you learned about the different areas you will interact with when working with Git: the working directory, the staging area, the commit history, and the local repository . We built a Git Diagram of these areas, and you ended the chapter by adding the first file to your rainbow project directory.

In this chapter, you will go through the process of making a commit in the Rainbow project and observe how each of the areas in the Git Diagram is involved. I will also introduce two important commands that will assist you in your day-to-day work with Git. The first command will allow you to check the state of your working directory and staging area, and the second will allow you to view a list of commits.

Current Setup

You now have a project directory called rainbow that has a .git directory inside it, and you also have one file, called rainbowcolors.txt, in your working directory. The staging area and commit history are empty because you have not yet made any commits in the rainbow repository. This can all be seen in Visualize it 3-1.

[VISUALIZE IT 3-1]



The rainbow project directory at the start of Chapter 3 has one untracked file in the working directory

Why Do We Make Commits?

In Chapter 2, you learned that a commit basically represents one version of a project. Every time you want to save a new version of a project, you can make a commit.

Committing is important because it allows you to back up your work and avoid the frustration of losing unsaved work. Once you've made a commit, that work is saved, and you'll be able to go back and look at that commit to see what your project looked like at that point in time.

In terms of when to make commits, there is no strict rule. This can depend on many factors, such as whether you're working on a project on your own or with other people, and what type of project you're working on (e.g., whether you're writing code that needs to compile or documentation for a feature). Finally, if you're working in a team, it may depend on the workflow your team uses and what conventions that team has agreed to (for example, how they use branches, which we'll discuss in Chapter 4).

A common adage in the world of Git is "commit early, commit often." If you're just getting started with Git, then I think this is good advice. At this stage, it is preferable to have too many commits than too few commits. Also, once you have a good grasp of Git basics, you can learn about additional tools that Git provides for cleaning up commits.

The Two Steps to Make a Commit

Now that we have touched upon why to commit, let's go over how to commit. Making a commit is a two-step process:

1. Add all the files you want to include in the next commit to the staging area.
2. Make a commit with a commit message.

Throughout this process you will be interacting with the four different areas of Git introduced in Chapter 2: the local repository, the working directory, the staging area, and the commit history.

A useful command in the committing process is the git status command. Among other things, it tells you the state of the working directory and the staging area. This is useful because in a project with many files it is easy to lose track of what state the files in your working directory are in (i.e., which ones you have edited) and which files you've added to the staging area.

[SAVE THE COMMAND]

git status

Show the state of the working directory and the staging area

In a project with many files, it can be hard to remember which files are untracked, which files are tracked, and which files you've edited. In the Rainbow project at the moment, you only have one file, but let's take a look at Example Book Project 3-1 to consider the case of a project with more files.

EXAMPLE BOOK PROJECT 3-1

The Book project I'm working on consists of 10 files, one for each chapter in the book. After working on the book for a while in between commits, I may forget which chapter files I have edited. I know I worked on chapters 2 and 3, but did I make any changes in chapter 4? If so, I don't want to lose them.

In this case, the git status command can help me by telling me which chapter files I've edited, which ones I've added to the staging area, and which ones I have yet to add.

The git status command only provides information; it never actually changes anything in your repository. Feel free to use it at any time while you're working on a Git project to learn more about the state of your working directory and staging area.

Go to Follow Along 3-1 to practice using the git status command and check on the state of the working directory and staging area in the rainbow repository. To carry out the steps in this Follow Along, you must be in the rainbow project directory in the command line, as indicated by the command prompt.

[FOLLOW ALONG 3-1]

1. rainbow $ **git status**
   On branch main
   No commits yet
   Untracked files:
   (use "git add <file>..." to include in what will be committed)
   rainbowcolors.txt
   nothing added to commit but untracked files present (use "git add" to track)

What to notice:

- The git status output informs you there are No commits yet. In other words, the commit history does not contain any commits at this time.
- The rainbowcolors.txt file is an untracked file.
- Git gives you the instructions that you need to add the untracked file to the staging area: use "git add <file>..." to include in what will be committed.

In Chapter 2, I mentioned that rainbowcolors.txt is an untracked file and that for it to become a tracked file it must be added to the staging area and included in a commit. So, let's carry out the first step in the process to make a commit, which is to add any files you want to include in the commit to the staging area.

ADDING FILES TO THE STAGING AREA

To add files to the staging area, you use the git add command. If you only want to add individual files that you have edited to the staging area, then you can pass in the file-name or filenames to the git add command as arguments. To add all the files you have edited or changed in your working directory, you can use the git add command with the -A option (which stands for "all"). This may be helpful if you have edited many files and don't want to write out each individual filename in the command line.

[SAVE THE COMMAND]

git add <filename>

Add one file to the staging area

git add <filename> <filename> ...

Add multiple files to the staging area

git add -A

Add all the files in the working directory that have been edited or changed to the staging area

As mentioned in Chapter 2, the staging area allows you to choose which updated files (or changes) will be included in your next commit. The general rule is to group related changes together. This allows you to keep your commits more organized. As you will see in the next section, every commit also has an associated commit message. This can be used to provide a description of what was updated in a specific commit.

This first step of the committing process allows you to be very specific about what you include in a commit. This means that you can edit many files in your project, but you

don't have to save them all in one commit. Let's explore this in Example Book Project 3-2.

EXAMPLE BOOK PROJECT 3-2

The Book project that I'm working on has 10 chapters represented by 10 files. Imagine a scenario where I work on chapters 1, 2, and 3 (therefore, I edit chapter_one.txt, chapter_two.txt, and chapter_three.txt).

If I decide that the work I have done on chapter 2 is ready to be committed (saved) but the work on chapters 1 and 3 is not something I want to include in my next commit, then I can add the updated version of the chapter 2 file to the staging area *without* adding the updated versions of the chapter 1 and chapter 3 files to the staging area. This means that *only* the changes in the chapter 2 file will be included in my next commit and officially "saved" in the local repository.

Example Book Project 3-2 illustrates how the staging area gives you a lot of control over what the saved versions of your project (your commits) look like.

In the Rainbow project, the rainbowcolors.txt file is the first file you will add to the staging area. This means that when you add this file, the index file (which represents the staging area in the .git directory) will be created. As you learned in Chapter 2, this file does not exist until you add a file to the staging area.The index file is a binary file, which means the actual contents look like gibberish to a human and are not easily understandable. For our purposes, we only need to understand that it represents the staging area. Go to Follow Along 3-2 to add the rainbowcolors.txt file to the staging area and create the index file.

[FOLLOW ALONG 3-2]

1. To see the index file being created, make sure to have a view of your rainbow project directory in the filesystem window with the hidden files and directories enabled and the contents of the .git directory visible.

2. rainbow $ **git add rainbowcolors.txt**

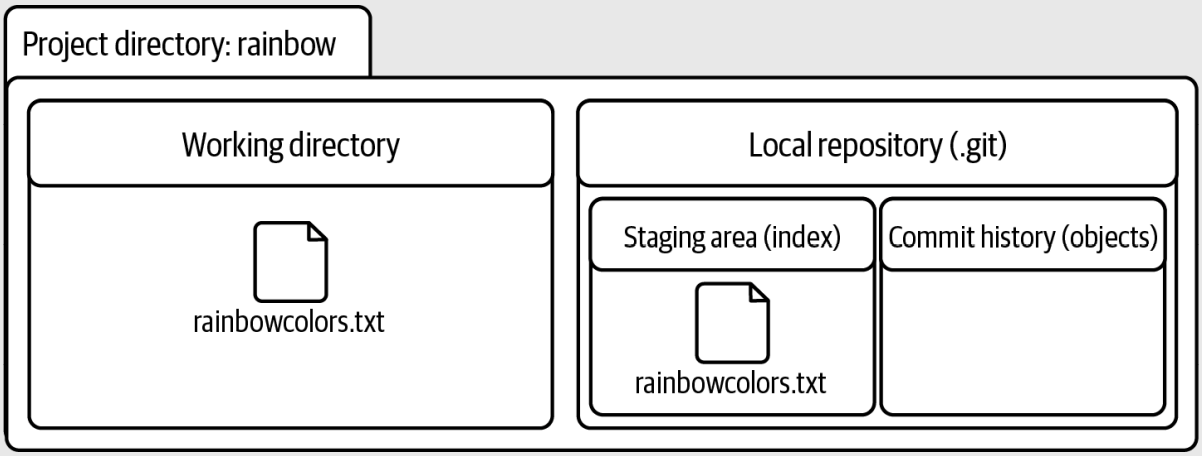
3. rainbow $ **git status**
   On branch main
   No commits yet
   Changes to be committed:
   (use "git rm --cached <file>..." to unstage)
   new file: rainbowcolors.txt

4. Using your filesystem window, look at the contents of the .git directory and identify the newly created index file.

What to notice:

- You added the rainbowcolors.txt file to the staging area, and in step 3 it is listed under the Changes to be committed: section. This is illustrated in Visualize It 3-2.

[VISUALIZE IT 3-2]



The rainbow project directory after you add the rainbowcolors.txt file to the staging area

The rainbowcolors.txt file is now both in the working directory and in the staging area. This is because the git add command does not *move* a file from the working directory to the staging area. It *copies* the file from the working directory into the staging area.

With the rainbowcolors.txt file in the staging area, you are now ready to move on to the second step in the committing process, which is to actually make a commit with a commit message.

MAKING A COMMIT

It's important to note that *commit* is both a verb and a noun. In Git, the verb *to commit* means to save something, and the noun (*a commit*) means a version of our project. So, to make a commit means to save a version of a project.

To make a commit, you will use the git commit command and pass in the -m option (which stands for "message"), typing in a message inside quotation marks. The message should usually be a brief description of the changes you made in this version of the project.

[SAVE THE COMMAND]

git commit -m "<message>"

Create a new commit with a commit message

Let's consider a sample commit message in Example Book Project 3-3.

EXAMPLE BOOK PROJECT 3-3

In my Book project, if I worked on chapter 2, added just that file to the staging area, and wanted to make a commit only with the updates to chapter 2, I might use the commit message "Updated chapter 2".

Keep in mind that different individuals and teams may have different rules about what to include in a commit message. If you're working on a Git project with other people, you should check with your collaborators so you're sure what they expect you to include in these messages.

For the Rainbow project, you will simply be passing in the name of the color you add to the project as the commit message, so that we can easily represent the commits in the Visualize It diagrams. Since the first color you added to the list of rainbow colors was red, in Follow Along 3-3 you will make a commit with the commit message "red".

[FOLLOW ALONG 3-3]

```
1 rainbow $ git commit -m "red"
  [main (root-commit) c26d0bc] red
  1 file changed, 1 insertion(+)
  create mode 100644 rainbowcolors.txt
```

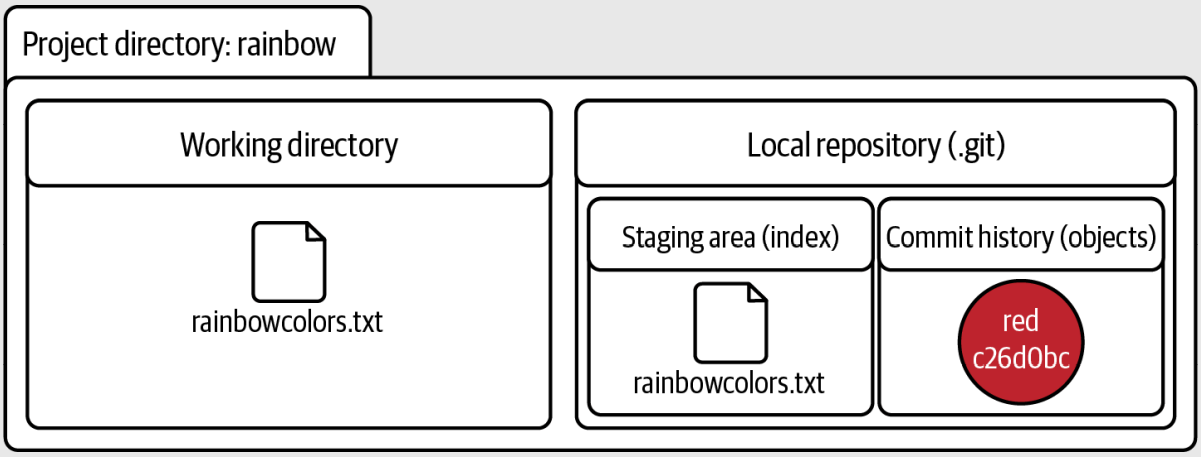
What to notice:

- The output of the git commit command shows the first seven characters of the commit hash for the red commit, which is c26d0bc in this book. The first seven

characters of your commit hash will be different, since, as you learned in "What Is a Commit?" on page 28, commit hashes are unique.

In Visualize it 3-3 you can see the red commit in the commit history, with the first seven characters of the commit hash.

[VISUALIZE IT 3-3]



The rainbow project directory after you make the red commit

You also learned in the previous chapter that when a new file that was untracked is added to the staging area and included in a commit, it becomes a tracked file, because Git now knows about it. Therefore, the rainbowcolors.txt file is now a tracked file.

The rest of the output following the git commit command is not really important for our purposes. It simply gives more information about what changed in this commit, and in my experience it's not something most Git users need to examine in detail.

Now that you've made the first commit in the rainbow repository, let's take a look at the information you can find about this commit in the commit history.

Viewing a List of Commits

To see a list of commits in the commit history, you use the git log command. The git log command lists the commits in a local repository in reverse chronological order. It displays four pieces of information about each commit:

1. Commit hash
2. Author name and email address
3. Date and time commit was made
4. Commit message

[SAVE THE COMMAND]

git log

Show a list of commits in reverse chronological order

[NOTE]

If the output of the git log command goes beyond the size of the command line window, to view the rest of the commits you must press Enter (Return) or use the down arrow. To exit the command, you must enter Q. Right now you have only one commit, so your git log output will be very short. However, as you add commits to the Rainbow project, your git log output will get longer and you will need to use these keys.

Go to Follow Along 3-4 to practice listing commits.

[FOLLOW ALONG 3-4]

1 rainbow $ **git log**

commit c26d0bc371c3634ab49543686b3c8f10e9da63c5 (**HEAD -> main**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:23:18 2022 +0100

red

What to notice:

- The git log output shows that at the moment you have only one commit, the red commit.

- The full commit hash of the red commit in this book is c26d0bc371c3634ab49543686b3c8f10e9da63c5. The commit hash in your project will be different.
- The author of the red commit should match the name and email address you provided in Chapter 1 when you set the user.name and user.email Git configuration variables.
- The date and time at which the commit was made are displayed.
- Below the date is the commit message, which in this case is red.

In the Visualize It diagrams, I will represent commits as circles. Each commit will be displayed in the color used in the commit message, and labeled with its full name or abbreviation (e.g., R for the red commit). Keep in mind that these circles each represent an individual commit that you can see listed in the git log output. This is made clear in Figure 3-1.

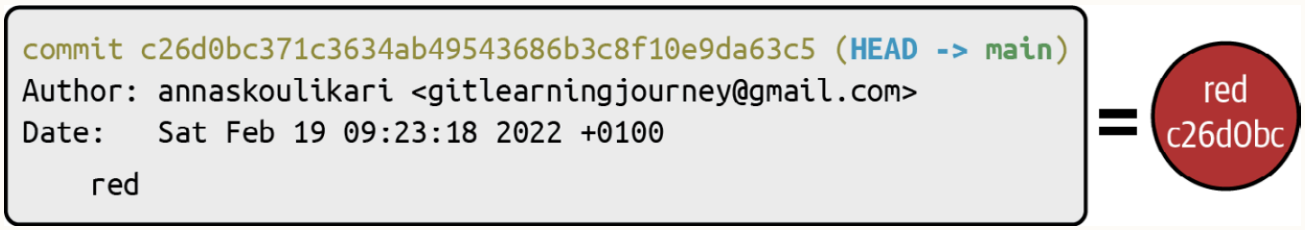


FIGURE 3-1

In this book, commits are represented by circles in the diagrams

There's one more thing that shows up in the git log output in Follow Along 3-4 that we have not covered yet. Next to the commit hash, inside the parentheses, you can see the text HEAD -> main. main is a branch; you'll learn what branches are and how to work with them in the next chapter, and you'll also learn what HEAD is.

Summary

This chapter introduced the two steps of making a commit, namely adding files to the staging area and making a commit with a commit message, and you went ahead and made your first commit in the rainbow repository. To assist in the process I introduced the git status command, which provides you with information about the state of the files in the working directory and staging area.

You also learned how to list the commits in a local repository using the git log command. In the git log output, you saw a mention of main in parentheses next to the commit hash.

In Chapter 4, you'll learn that main is a *branch*, and we will explore why and how branches are used in Git.

Previous chapter

‹ Chapter 2. Local Repositories

Next chapter

Chapter 4. Branches ›